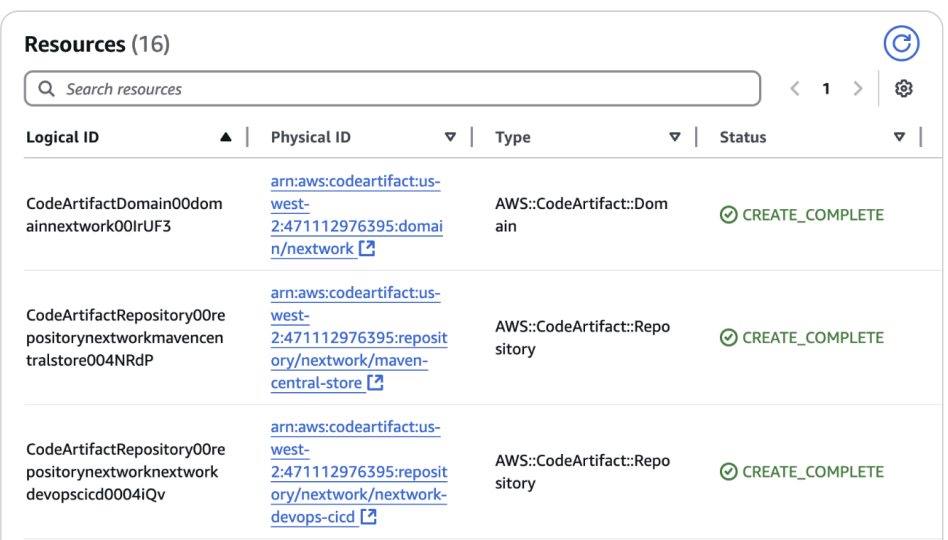
nextwork.org

Infrastructure as Code with CloudFormation

CH Arpita Chowdhury



Resources (16)

Search resources

1

Logical ID	Physical ID	Type	Status
CodeArtifactDomain00domainnextwork00IrUF3	arn:aws:codeartifact:us-west-2:471112976395:domain/nextwork	AWS::CodeArtifact::Domain	CREATE_COMPLETE
CodeArtifactRepository00repositorynextworkmavencentralstore004NRdP	arn:aws:codeartifact:us-west-2:471112976395:repository/nextwork/maven-central-store	AWS::CodeArtifact::Repository	CREATE_COMPLETE
CodeArtifactRepository00repositorynextworknextworkdevopscicd0004iQv	arn:aws:codeartifact:us-west-2:471112976395:repository/nextwork/nextwork-devops-cicd	AWS::CodeArtifact::Repository	CREATE_COMPLETE

CH
Arpita Chowdhury
NextWork Student
nextwork.org

Introducing Today's Project!

In today’s project, I will demonstrate how to rebuild a full CI/CD pipeline using AWS CloudFormation so the same infrastructure I usually click together in the console (dev instance, CodeDeploy app + deployment group, and all supporting IAM roles/policies) can be created repeatably, consistently, and faster from a single template. I’m doing this project to learn how “infrastructure as code” works in real life: how to define resources with dependencies, avoid manual configuration drift, and make deployments more reliable by version-controlling the exact setup. By the end, I’ll have a CloudFormation template I can re-use for future projects (or different environments like dev/test/prod), proving I can build a CI/CD foundation in a way that’s scalable and production-aligned.

Key tools and concepts

Services I used were AWS CodePipeline, AWS CodeBuild, AWS CodeDeploy, Amazon EC2, Amazon S3, AWS CodeArtifact, AWS CloudFormation, AWS IAM, and GitHub. Key concepts I learnt include end-to-end CI/CD pipeline design, source-to-production automation, artifact management with CodeArtifact and S3, IAM roles and least-privilege permissions, service integrations across AWS Developer Tools, deployment strategies with CodeDeploy, rollback and recovery handling, and how infrastructure-as-code with CloudFormation enables repeatable, reliable DevOps workflows.

Project reflection

This project took me approximately 6 hours. The most challenging part was debugging permission and deployment failures across multiple AWS services, especially identifying IAM and S3 access issues between CodePipeline, CodeBuild, CodeDeploy, and the EC2 instance.

CH

Arpita Chowdhury
NextWork Student

nextwork.org

It was most rewarding to see the full CI/CD pipeline run successfully end-to-end, automatically deploying my code to production and then confidently triggering a rollback and watching the application revert correctly, proving that the system is reliable, automated, and production-ready.

This project is part six of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project Build a CI/CD Pipeline with AWS.

CH
Arpita Chowdhury
NextWork Student
nextwork.org

Generating a CloudFormation Template

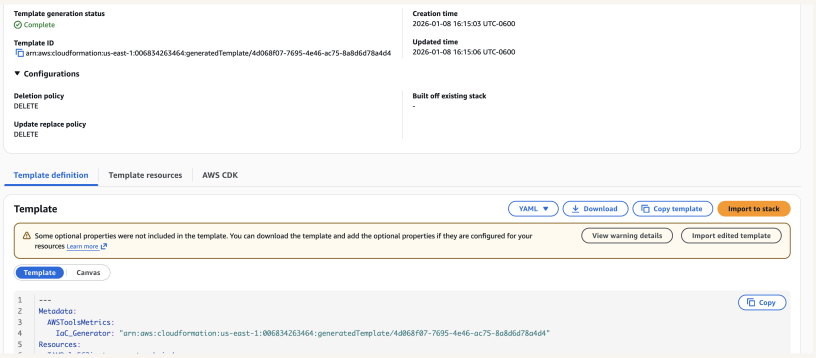
The IaC Generator is an AWS CloudFormation feature that helps you automatically create Infrastructure-as-Code templates from resources you've already built (or can discover) in your AWS account, so you can convert "console-built" infrastructure into a reusable CloudFormation template. It works in a three-step process where: Scan / Discover resources – CloudFormation identifies the existing AWS resources you want to turn into IaC. Generate the template – It produces a CloudFormation template (usually YAML/JSON) that represents those resources and their configurations. Review & refine – You validate the generated template, adjust anything missing or not modeled perfectly (like parameters, outputs, names, or dependencies), then save it for repeatable deployments.

A CloudFormation template is a declarative Infrastructure-as-Code file (written in YAML or JSON) that describes all the AWS resources you want to create, how they're configured, and how they relate to each other so AWS can automatically provision, update, or delete the entire infrastructure as a single stack. The resources that I could add to my template include core components of my CI/CD and web app setup, such as: EC2 instances (for the development or application server) IAM roles and instance profiles (to grant EC2 and CodeDeploy the permissions they need) AWS CodeDeploy resources (applications and deployment groups) Security groups (to control inbound and outbound traffic) S3 buckets (to store application artifacts or deployment bundles) Auto Scaling groups and launch templates (if scaling is required later) By defining these resources in a CloudFormation template, I can deploy the same infrastructure consistently across environments and avoid manual setup errors.

CH

Arpita Chowdhury
NextWork Student

The resources I couldn’t add to my template were the CodeBuild project and the CodeDeploy deployment group, because they require detailed, context-specific configuration and sensitive permissions (such as build environment settings, service roles, artifact locations, and deployment hooks) that the IaC Generator can’t safely or accurately infer automatically. While these resources are fully supported by CloudFormation, they need to be manually defined in the template to ensure the correct security roles, environment variables, and deployment behavior are configured properly.

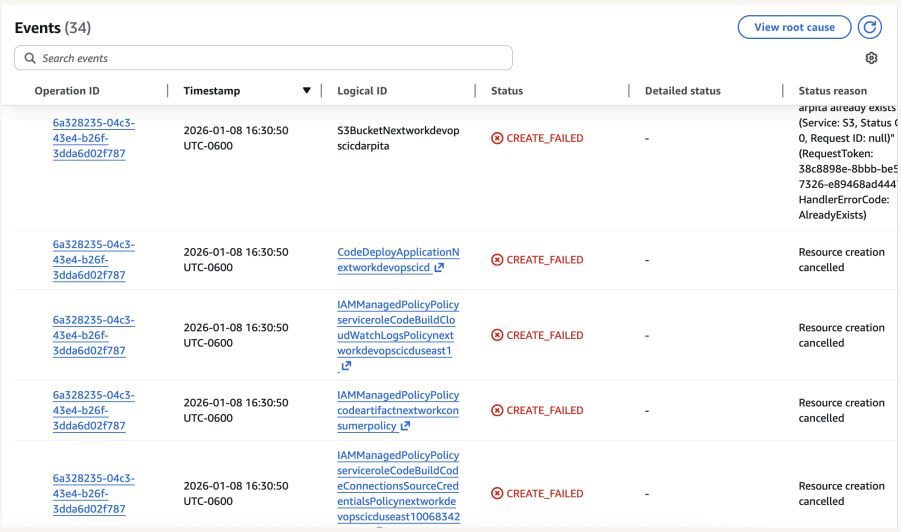


CH
Arpita Chowdhury
NextWork Student

Template Testing

Before testing my template, I deleted all existing CI/CD resources from my AWS account including the CodeDeploy application, CodeBuild project, CodeArtifact domain and repository, S3 bucket, EC2 development instance, and all related IAM roles and policies because CloudFormation cannot create resources with names that already exist, and removing them ensures a clean, conflict-free environment to accurately verify that my CloudFormation template can fully recreate the CI/CD infrastructure from scratch.

I tested my template by deploying it as a new CloudFormation stack after deleting my existing CI/CD resources. The result of my first test was a failed stack creation, because some resources (like the S3 bucket and certain IAM policies) already existed with the same names, causing CloudFormation to stop the deployment and cancel the remaining resource creation.



CH
Arpita Chowdhury
NextWork Student

DependsOn

To resolve the error, I updated my CloudFormation template to explicitly control the resource creation order by adding a DependsOn attribute, ensuring that IAM roles were created before any managed policies attempted to attach to them. This prevented CloudFormation from trying to reference resources that didn't exist yet and allowed the stack to progress successfully. The DependsOn attribute means that CloudFormation must fully create the specified resource first before creating the current one, enforcing a strict dependency order and avoiding race conditions during stack deployment.

The DependsOn line was added to four different parts of my template: each of the four CodeBuild IAM managed policies (the ones that start with IAMManagedPolicy...policyservicerole...). I added DependsOn there to make CloudFormation create the CodeBuild IAM role first before it tries to attach those policies to it. For the CodeArtifact policy, I updated DependsOn to wait for the EC2 instance IAM role (the role that grants CodeArtifact access to my dev EC2 instance), and since that same policy can also be attached to the CodeBuild role, I set it to depend on both roles so CloudFormation doesn't try to attach the policy before the roles exist.

```
71  IAMManagedPolicy00policyserviceroleCodeBuildCodeConnectionsSourceCreden
72    UpdateReplacePolicy: "Delete"
73    Type: "AWS::IAM::ManagedPolicy"
74    DeletionPolicy: "Delete"
75    DependsOn: "IAMRole00codebuildnextworkdevopscicdservicerole00Bd4sy"
76    Properties:
```

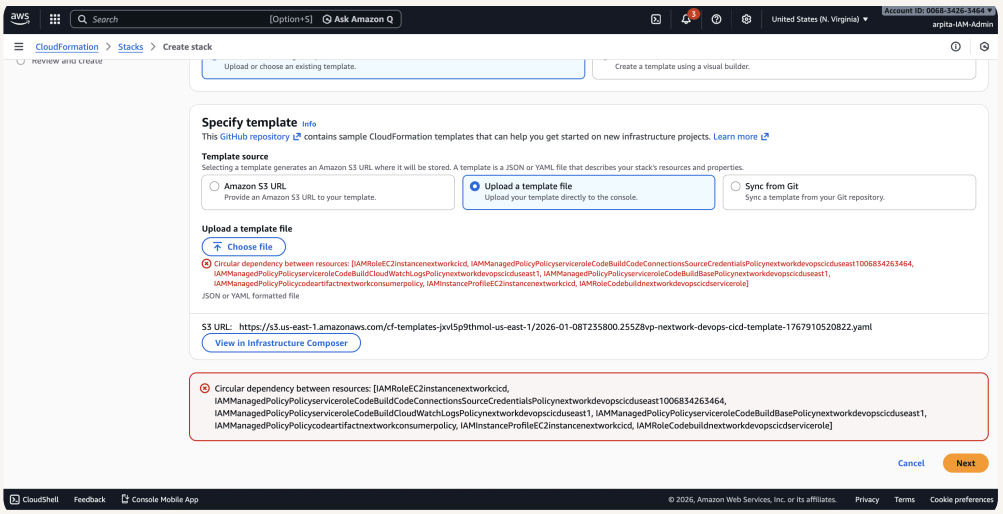
CH

Arpita Chowdhury
NextWork Student

Circular Dependencies

I gave my CloudFormation template another test! But this time, CloudFormation wouldn’t even let me proceed past the upload step it immediately threw a circular dependency error, because my IAM resources were referencing each other in a loop (the IAM roles and managed policies were both trying to attach to each other, and my DependsOn lines made that dependency cycle explicit), so CloudFormation couldn’t figure out a valid order to create them.

To fix this error, I edited my CloudFormation template to remove the ManagedPolicyArns references from the IAM roles (especially the CodeBuild role), because those role-to-policy references combined with the policies attaching back to the roles created a circular dependency that CloudFormation couldn’t resolve during stack creation.



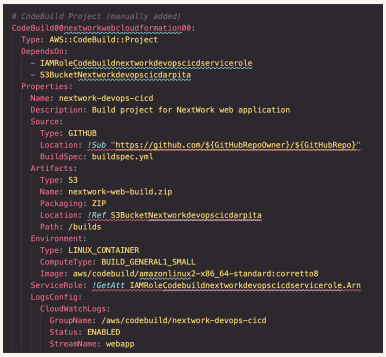
Manual Additions

In a project extension, I manually defined two more resources: an AWS CodeBuild Project (to pull my code from GitHub, run the buildspec steps, and output build artifacts into my S3 bucket with CloudWatch logging enabled) and an AWS CodeDeploy Deployment Group (to connect my CodeDeploy application to the EC2 instances tagged as webservers and control how deployments are rolled out).

I also had to make sure the references were consistent in this template, so I edited the placeholder IDs in my manually-added resources to match the exact logical resource IDs already defined in my IaC-generated template specifically replacing <YOUR_CODEBUILD_SERVICE_ROLE_ID> with IAMRoleCodebuildnextworkdevopscicdservicerole, <YOUR_S3_BUCKET_ID> with S3BucketNextworkdevopscicdarpita, <YOUR_CODEDEPLOY_ROLE_ID> with IAMRoleNextWorkCodeDeployRole, and <YOUR_CODEDEPLOY_APPLICATION_ID> with CodeDeployApplicationNextworkdevopscicd.

I also introduced Parameters, which are dynamic inputs in a CloudFormation template that let you customize values at deployment time (such as a GitHub repository owner or name) instead of hard-coding them making the template more reusable, flexible, and production-ready across different environments or projects.

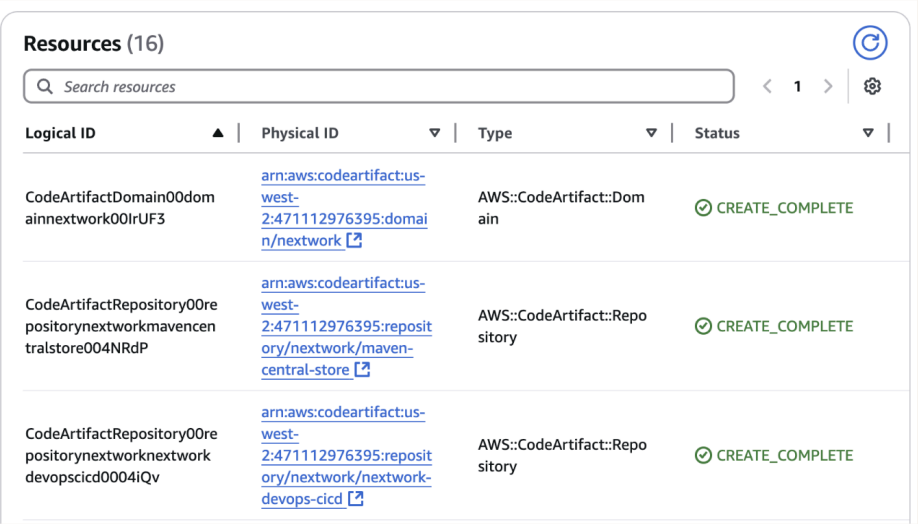
Arpita Chowdhury
NextWork Student



CH
Arpita Chowdhury
NextWork Student

Success!

I could verify all the deployed resources by visiting the AWS CloudFormation console, opening my stack, and checking the Resources tab, where I could clearly see each provisioned service (such as CodeArtifact domains and repositories) listed with their physical IDs, resource types, and CREATE_COMPLETE status, confirming that the infrastructure was successfully deployed and managed by CloudFormation.



The place to learn & showcase your skills

Check out nextwork.org for more projects